

Aura: A Deterministic Commitment-Native Protocol for Canonical Identity, Verifiable Execution, and Fail-Closed Settlement

Tyler McRae
Independent Research

April 2026

Abstract

Aura is a deterministic commitment-native protocol that collapses identity, execution, proof binding, authorization, and settlement into a single canonical object pipeline. The protocol is built around one non-negotiable identity surface, `AURA_HASH_V1`, one fixed field modulus $p = 2^{521} - 1$, one deterministic execution recurrence, one ordered trace commitment, one canonical proof envelope boundary, one canonical authorization lineage encoding, and one minimal settlement reference object. The system is designed to eliminate representation drift rather than merely detect it after the fact.

The core claim of Aura is not simply that valid computations can be proven, but that for every valid canonical artifact there exists exactly one valid upstream interpretation inside the active protocol boundary. This is achieved through strict byte-level canonicalization, exact domain separation, fail-closed wire validation, rejection of parallel representations, and derivation of settlement linkage from prior head state rather than caller-supplied linkage fields. A derived human-readable identity surface, `UDOT`, is included as a deterministic presentation layer while remaining outside the primary cryptographic commitment boundary.

This paper formalizes the active Aura pipeline, defines its main functions and invariants, and analyzes the security consequences of enforcing a single-path deterministic protocol architecture. The result is a system aimed at reproducibility, verifiability, and ambiguity resistance across both implementation and transport layers.

1. Introduction

Many cryptographic systems are formally secure at the primitive level while still remaining operationally ambiguous at the protocol level. The source of this ambiguity is often not the weakness of a hash function, a field, or a proof system in isolation, but the existence of multiple accepted encodings, multiple identity surfaces, optional compatibility layers, alternate nested objects, or settlement interfaces that allow caller-controlled reinterpretation of state. Under such designs, two artifacts may be semantically intended to represent the same object while differing in encoding, normalization profile, stage ordering, or reference structure. Even where these differences are later checked, the system has already admitted multiple potential paths through the pipeline.

Aura is designed around the opposite objective. Rather than allowing multiple admissible representations and reconciling them with equivalence checks, Aura removes such degrees of freedom from the canonical path. It enforces a single identity surface, a single deterministic execution model, a single trace commitment rule, a single proof-bound claim representation, a single authorization lineage encoding, a single artifact derivation chain, and a single final settlement reference object.

Failures are typed and fail closed; non-canonical parallel representations are excluded instead of tolerated.

The protocol is therefore commitment-native in a precise sense. It does not begin with a general execution system and later attach commitments as auxiliary metadata. Instead, commitment formation defines the pipeline itself. The request is canonicalized. The resulting bytes are hashed under a single identity contract. Deterministic derivation functions map canonical byte surfaces into field elements and recurrence parameters. The resulting state trace is committed in order. Proof binding is defined only with respect to the canonical storm claim and its compact public inputs. Authorization references the proof by its canonical hash. Settlement references upstream state only by the canonical proof hash. Continuous head linkage is derived from prior state rather than accepted as free input.

This paper develops that architecture in full. Section 2 states the protocol problem Aura addresses. Section 3 summarizes the active pipeline. Sections 4 through 9 define the core mathematical and structural components: canonical hashing, field arithmetic, storm derivation, recurrence, trace layout, commitment, and proof binding. Sections 10 through 12 define artifact derivation, authorization, local burn semantics, settlement-wire minimality, and continuous head progression. Section 15 describes the derived glyph representation layer. Section 17 states core invariants. Section 19 analyzes the security consequences of the design. The final sections discuss failure classes, implementation consequences, and the conceptual position of Aura as a single-path protocol.

2. Motivation and Problem Statement

A protocol can be cryptographically conservative while remaining semantically permissive. In practice, several recurring design habits create ambiguity:

1. **Multiple identity surfaces.** A payload might be hashed directly in one place, framed and hashed in another, normalized as text in a third, or wrapped into larger preimage objects elsewhere.
2. **Parallel claim representations.** Proof objects may embed both legacy and current claims, or may mix full upstream objects with partial references.
3. **Weak stage ownership.** If multiple documents or code paths redefine the same boundary, the system admits drift in how operators interpret it.
4. **Loose settlement linkage.** When previous-head hashes or sequence numbers are caller-supplied instead of derived, settlement admits representational freedom at its most authoritative stage.
5. **Presentation promoted to truth.** Human-readable artifacts are useful, but if they are accepted as equivalent canonical representations of cryptographic state, the system becomes overexposed to formatting ambiguity.

Aura treats these not as convenience issues but as protocol-level risks. The goal is therefore to enforce what may be called a *single-path system*: a system in which every active canonical concept has exactly one owning representation at each stage, and every downstream stage refers to upstream state through exactly one canonical reference surface. The protocol should not merely detect alternate forms; it should exclude them from the canonical path.

Definition 2.1 (Single-Path Protocol). *A protocol is single-path if, for every accepted canonical artifact, there exists exactly one valid upstream canonical interpretation under the active stage ordering, field encoding, derivation rules, and wire contracts.*

Aura aims to instantiate this definition concretely.

3. System Overview

Aura is organized as an ordered canonical pipeline. The active stage map is:

```
request → proof_material_hash → proof_hash → udot_bundle_v2 → authorization →
storm → trace → proof → settlement
```

The architecture may be summarized in layered terms as follows.

3.1. Canonical Layer Roles

- **Identity layer.** AURA_HASH_V1 defines the sole canonical identity function for message bytes.
- **Field and derivation layer.** The protocol operates over the prime field $\mathbb{F}_p$ with modulus $p = 2^{521} - 1$. Domain-separated derivation functions map canonical byte surfaces to field elements and recurrence material.
- **Execution layer.** STORM_V1_1 deterministically transforms canonical input bytes into an ordered state trace.
- **Commitment layer.** The trace is committed by an ordered Merkle reduction to TRACE_ROOT.
- **Proof layer.** The canonical proof envelope binds exactly one canonical storm claim and exactly one proof hash.
- **Artifact layer.** Proof material, fractal key derivation, proof hash derivation, and UDOT derivation form an exact chain.
- **Authorization layer.** Canonical subject, intent, and freshness bindings reference the proof only via its canonical proof hash.
- **Settlement layer.** The final canonical settlement object carries only a minimal proof reference, while local settlement accounting and continuous head linkage remain deterministic and fail closed.

3.2. Design Philosophy

The protocol prioritizes three principles.

Canonical exclusivity. Every active concept has one canonical representation and one owner. Cross-reference is allowed; duplicated definition is not.

Pure stage outputs. Each canonical stage emits exactly one artifact with no optional fields, no null values, and no alternate encodings within the active path.

Derived authority. Later authority is constructed from earlier canonical state rather than supplied independently. This is most visible in continuous settlement linkage, where previous-head and sequence values are derived from the prior head instead of being admitted as free input.

4. Canonical Identity Surface

Aura begins with one identity function and only one. This is the first and most important lock.

Definition 4.1 (Canonical Message Bytes). *Let m denote an arbitrary byte string. The canonical message bytes are*

$$\text{cmb}(m) = \text{u64_le}(|m|) \parallel m,$$

where $\text{u64_le}(|m|)$ is the exact unsigned 64-bit little-endian encoding of the byte length of m .

Definition 4.2 (Canonical Hash Function). *The sole canonical identity function is*

$$\text{AURA_HASH_V1}(m) = \text{SHA256}(\text{"AURA_HASH_V1"} \parallel \text{cmb}(m)).$$

The consequences are immediate.

- The length prefix is always present.
- The prefix is always unsigned 64-bit little-endian.
- No lower layer may add framing, alternate prefixes, alternate hashes, or alternate normalization.
- Text mode, when used, is a helper profile for deriving bytes before hashing; it is not a second identity function.

4.1. Text Profile

If a caller starts from text, Aura permits only a narrow exact profile:

1. Decode UTF-8.
2. Normalize to NFC.
3. Replace `\r\n` with `\n`.
4. Replace `\r` with `\n`.
5. Reject byte order marks corresponding to `U+FEFF`.
6. Re-encode UTF-8.
7. Pass the resulting bytes into `AURA_HASH_V1`.

Whitespace is preserved. Trimming is invalid. Invalid UTF-8 and BOM-containing text are rejected.

Theorem 4.1 (Identity Uniqueness). *For every accepted canonical message byte string m , there exists exactly one canonical message root*

$$\text{MESSAGE_ROOT} = \text{AURA_HASH_V1}(m).$$

Proof. The framing bytes are fixed, the domain separator is fixed, and the underlying hash function is deterministic. No alternate canonical framing or normalization path is admissible. Therefore exactly one canonical preimage is accepted for each accepted byte string, and thus exactly one canonical message root is produced. $\square$

5. Field Arithmetic and Encoding

Aura operates over the prime field

$$\mathbb{F}_p = \mathbb{F}_p, \quad p = 2^{521} - 1.$$

The canonical field surface is deliberately narrow.

Definition 5.1 (Canonical Field Element Encoding). *A field element is encoded as exactly 66 bytes in big-endian order. The encoding is valid only when:*

1. *the top seven bits of byte 0 are zero;*
2. *the encoded integer value is strictly less than p ;*
3. *the byte string is canonical.*

The protocol admits the following operations as canonical field operations:

$$a + b \pmod{p}, \quad a - b \pmod{p}, \quad ab \pmod{p}, \quad a^2 \pmod{p}.$$

Arbitrary byte strings may additionally be reduced by repeated radix-256 folding, but no alternate modulus and no alternate byte width are valid.

5.1. Why the Field Boundary Matters

In many systems, field encodings are treated as mere implementation detail. Aura instead promotes encoding minimality to a protocol boundary. This matters for two reasons.

First, proof and execution surfaces must agree on exact byte-level meaning. If one implementation allows non-canonical top-bit patterns or alternative padded widths, another implementation may produce the same numeric state while disagreeing on the canonical byte identity of that state. Aura forbids that disagreement.

Second, the field width aligns directly with other protocol surfaces. Storm rows encode state as a pair of canonical 66-byte field encodings. That exact width propagates into row bytes, leaf formation, and ultimately `TRACE_ROOT`. Canonical field encoding is therefore not ancillary; it is part of the commitment boundary.

Proposition 5.1 (Field Encoding Determinism). *Every accepted field element has exactly one accepted canonical byte encoding.*

Proof. The width is fixed at 66 bytes, the endianness is fixed, the top-bit constraint is fixed, and out-of-range values are rejected. Therefore no second accepted canonical encoding exists for the same field element. $\square$

6. Domain-Separated Derivation Functions

Aura distinguishes the raw message identity surface from the storm execution derivation surface. The bridge is not a free conversion but a fixed family of domain-separated functions.

6.1. The 521-bit Hash Lifting Function

Define $\text{AURA_HASH521_V1}(x)$ as follows:

1. Compute

$$h_0 = \text{SHA3-512}(x \parallel 0x00), \quad h_1 = \text{SHA3-512}(x \parallel 0x01).$$

2. Take 512 bits from h_0 and the first 9 bits of h_1 , most-significant-bit first.
3. Pack the resulting 521 bits into a 66-byte big-endian value.
4. If the packed value equals $2^{521} - 1$, map it to zero.

This function is then used with exact ASCII domain separators to derive the initial state and recurrence coefficients from raw input bytes.

6.2. Canonical Storm Derivations

Let `side_A`, `side_B`, and `context_bytes_v1` denote the canonical storm input byte surfaces. Then:

$$x_0 = \text{AURA_HASH521_V1}(\text{"AURA_X0_V1"} \parallel \text{side_A}),$$

$$y_0 = \text{AURA_HASH521_V1}(\text{"AURA_Y0_V1"} \parallel \text{side_B}),$$

$$a = \text{AURA_HASH521_V1}(\text{"AURA_C_A_V1"} \parallel \text{context_bytes_v1}),$$

$$b = \text{AURA_HASH521_V1}(\text{"AURA_C_B_V1"} \parallel \text{context_bytes_v1}),$$

and for each iteration index n ,

$$\phi_n = \text{AURA_HASH521_V1}(\text{"AURA_STORM_X_V1"} \parallel \text{side_A} \parallel \text{side_B} \parallel \text{context_bytes_v1} \parallel \text{u64_le}(n)),$$

$$\psi_n = \text{AURA_HASH521_V1}(\text{"AURA_STORM_Y_V1"} \parallel \text{side_A} \parallel \text{side_B} \parallel \text{context_bytes_v1} \parallel \text{u64_le}(n)).$$

These formulas do not admit alternate separators or alternate step encodings. Their exactness has two benefits. First, it separates roles unambiguously: `AURA_HASH_V1` governs message identity, while the 521-bit derivation family governs field lifting and execution. Second, it prevents cross-surface collisions caused by ad hoc reuse of raw hash bytes.

7. Deterministic Storm Execution

The active execution function is `STORM_V1_1`. It accepts exactly:

- `side_A`: 110 bytes,
- `side_B`: 110 bytes,

- `context_bytes_v1`: 209 bytes,
- `iteration_count`: u64.

The context bytes are valid only when their version and execution-domain bytes match the fixed contract. No optional fields, shortened contexts, or alternate domain bytes are valid inside the canonical path.

7.1. State Definition

The initial state is

$$s_0 = (x_0, y_0) \in \mathbb{F}_p \times \mathbb{F}_p.$$

For each $n \in \{0, \dots, N - 1\}$ with $N = \text{iteration_count}$, define

$$x_{n+1} = x_n^2 - y_n^2 + a + \phi_n \pmod{p},$$

$$y_{n+1} = 2x_n y_n + b + \psi_n \pmod{p}.$$

The ordered canonical trace is

$$T = [s_0, s_1, \dots, s_N].$$

7.2. Determinism Property

Theorem 7.1 (Storm Determinism). *For fixed canonical storm inputs `side_A`, `side_B`, `context_bytes_v1`, and `iteration_count`, the values*

$$s_0, s_N, T$$

are uniquely determined.

Proof. The derivation functions for x_0 , y_0 , a , b , ϕ_n , and ψ_n are all deterministic and domain-separated. The recurrence itself is deterministic over a fixed modulus. Therefore each state is uniquely determined from the previous state and canonical inputs, yielding a unique ordered trace. $\square$

7.3. Why This Matters

The point of determinism here is deeper than reproducibility. In a system with optional pre-processing, inference of omitted rows, or alternate recurrence encodings, two implementations could agree on the end state while disagreeing on the path that led there. Aura rejects that design. The ordered trace itself is part of the committed object, and no row may be removed, inferred, or reordered.

8. Trace Layout and Ordered Commitment

Aura makes the trace byte-exact before it makes it cryptographic.

8.1. Row Layout

Each state row is defined as

$$\text{row_bytes}_n = x_n^{(66, \text{be})} \parallel y_n^{(66, \text{be})},$$

where each field value is encoded canonically as a 66-byte big-endian field element. Therefore every row is exactly 132 bytes.

The row order is exact:

$$\text{row}_0, \text{row}_1, \dots, \text{row}_N.$$

The initial state is carried in row 0, and the final state is carried in row N .

8.2. Merkle Commitment Rule

Leaves are defined by

$$\ell_n = \text{SHA3-256}(\text{row_bytes}_n).$$

Parent nodes are defined by

$$\text{parent}(L, R) = \text{SHA3-256}(L \parallel R).$$

If a level has odd cardinality, the last node is duplicated before parent hashing. No domain separators are inserted at the leaf or parent layer. No sorting is allowed. No zero padding is allowed.

The final root is `TRACE_ROOT`.

Definition 8.1 (Canonical Trace Commitment). *TRACE_ROOT is the final 32-byte root produced by ordered SHA3-256 Merkle reduction of the full row sequence.*

Theorem 8.1 (Order Binding). *If two accepted traces differ in row content or row order, then their canonical TRACE_ROOT values differ except in the event of a hash collision.*

Proof. Leaves are hashed from exact row bytes in exact order. The parent rule preserves order via concatenation of left and right child hashes. Any change in a row or in the sequence of rows changes at least one leaf or one parent input, hence the root, unless a hash collision occurs. $\square$

The duplicate-last odd-level rule is protocol-significant. It eliminates implementation freedom around odd-width tree handling. That choice is not cosmetic; it is part of the active commitment contract.

9. Proof-Bound Claim and Compact Public Inputs

Aura separates proof production from proof transport, while still enforcing a single canonical proof-bound claim.

9.1. Canonical Storm Claim

The canonical storm claim carries:

- `version`,
- `modulus_id`,

- iteration_count,
- side_A,
- side_B,
- context_bytes_v1,
- initial_state,
- final_state,
- TRACE_ROOT.

It must not carry legacy commitment aliases or compatibility names for TRACE_ROOT.

9.2. Compact Public Inputs

The compact public inputs are:

- version,
- modulus_id,
- iteration_count,
- side_a_hash,
- side_b_hash,
- context_hash,
- initial_state,
- final_state,
- TRACE_ROOT.

The hashes are:

$$\begin{aligned} \text{side_a_hash} &= \text{SHA3-256}(\text{"AURA_STORM_SIDE_A_HASH_V1"} \parallel \text{side_A}), \\ \text{side_b_hash} &= \text{SHA3-256}(\text{"AURA_STORM_SIDE_B_HASH_V1"} \parallel \text{side_B}), \\ \text{context_hash} &= \text{SHA3-256}(\text{"AURA_STORM_CONTEXT_HASH_V1"} \parallel \text{context_bytes_v1}). \end{aligned}$$

9.3. Proof Envelope Boundary

The canonical external proof object is the envelope:

- proof_version,
- proof_session_id_hex,
- proof_hash_hex,

- `storm_claim`.

Unexpected fields are invalid. Legacy claims, authorization objects, or other upstream parallel objects are not allowed inside the canonical proof envelope.

Theorem 9.1 (Single Claim Representation). *Inside the canonical proof boundary, there exists exactly one proof-bound claim representation: the canonical storm claim.*

Proof. By protocol contract, the canonical envelope permits exactly one claim field, namely `storm_claim`, and explicitly excludes parallel claim representations and legacy aliases. Therefore no accepted proof envelope may carry two simultaneous canonical claim interpretations. $\square$

This is a central design choice. Many systems permit coexistence of raw claim bytes, normalized claims, embedded authorization, and multiple compatibility fields. Aura forbids that coexistence on the canonical wire.

10. Artifact Derivation Chain

Aura next binds proof material, subject identity, freshness, and human-readable derivation into one exact chain.

10.1. Proof Material

`ProofMaterialV1` binds three ordered components:

1. `SHA256(proof_blob_bytes)`,
2. `SHA256(public_inputs_bytes)`,
3. `SHA256(verification_key_bytes)`.

Let `proof_material_bytes` denote the canonical serialization of that object. Then

`proof_material_hash = SHA256("AURA_PROOF_MATERIAL_V1" || proof_material_bytes)`.

10.2. Fractal Key and Proof Hash

`FractalKeyV1` consists of exactly three ordered components:

1. subject binding,
2. challenge binding,
3. proof material hash.

Let `fractal_key_bytes` be the canonical serialization of this object. Then:

`proof_hash = SHA256("AURA_FRACTAL_KEY_V1" || fractal_key_bytes)`.

This derived proof hash is the canonical upstream proof reference for later pipeline stages.

10.3. Exact Chain

The artifact chain is therefore:

`proof blob/public inputs/VK → ProofMaterialV1 → proof_material_hash → FractalKeyV1 → proof_hash`

Proposition 10.1 (No Branching in Artifact Derivation). *Within the active canonical path, there is exactly one downstream proof reference surface, namely `proof_hash_hex`.*

Proof. By contract, downstream canonical objects reference upstream artifacts principally by `proof_hash_hex`. The final settlement wire carries only this reference, and UDOT is derived from it rather than existing as an independent canonical identity alias. $\square$

11. Authorization Lineage

Authorization in Aura is not an open-ended metadata object. It is a fixed canonical binding of subject, intent, freshness, and proof reference.

11.1. Canonical Authorization Envelope

`AuthorizationIntentEnvelopeV1` contains:

- `intent_version`,
- `intent_id_hex`,
- `proof_hash_hex`,
- `authorization_lineage`.

The lineage itself is fixed to six fields:

- `subject_binding_type` = `submitter-pubkey-base58`,
- `subject_binding`,
- `intent_type` = `opaque-intent-hash-32`,
- `intent_commitment_hex` = `intent_id_hex`,
- `freshness_binding_type` = `challenge-pubkey-base58`,
- `freshness_binding`.

Unexpected fields are invalid. The object must not embed submission requests, proof envelopes, or settlement objects.

11.2. Interpretation

This lineage contract does two things simultaneously.

First, it binds proof to identity and freshness without copying upstream objects. This preserves the purity rule: downstream stages should reference upstream objects by canonical identifiers rather than carry nested duplicates.

Second, it enforces a single encoding. Aura rejects the tempting but dangerous pattern of admitting multiple shape-compatible lineage objects and then comparing them semantically. The protocol admits one canonical lineage shape only.

Theorem 11.1 (Authorization Single-Encoding Property). *There exists exactly one accepted canonical authorization lineage encoding inside the active pipeline.*

Proof. The lineage field list is fixed and exact. The contract explicitly excludes alternate lineage encodings and unexpected fields. Therefore any accepted canonical authorization lineage instance must instantiate that one six-field form. $\square$

12. Minimal Final Settlement Object

The final settlement wire is deliberately minimal. This is not because the protocol lacks structure upstream, but because settlement should carry only the reference it needs.

12.1. Final Object

`SolanaSettlementRequestWireV1` contains exactly:

- `settlement_version`,
- `solana_rpc_url`,
- `commitment_config`,
- `proof_hash_hex`.

The rules are:

- `settlement_version` must be `v1`,
- `solana_rpc_url` must be present and not null,
- `commitment_config` must be one of `processed`, `confirmed`, or `finalized`,
- `proof_hash_hex` must be canonical lowercase 64-hex,
- unexpected fields are invalid.

12.2. Minimality as a Security Property

The final object must not embed:

- the proof envelope,
- the authorization intent,
- the UDOT bundle,
- or any other upstream canonical object.

This rule has structural value. By the time the pipeline reaches settlement-wire construction, canonical truth about the upstream pipeline has already been compressed into one canonical proof reference. Carrying additional upstream objects in the final settlement request would create room for redundant interpretations or mismatch checks at the most external boundary. Aura refuses that redundancy.

13. Ledger, Burn, and Fail-Closed Economics

Aura distinguishes the minimal final settlement wire from the local economic and accounting rules exercised by the local settlement fixtures. These local rules remain part of the protocol design and are deterministic.

13.1. Ledger Validity

A local ledger is valid only when:

1. `ledger_policy_version = 1`,
2. accounts are ordered,
3. account identifiers are unique,
4. the payer account exists,
- 5.

$$\sum_i \text{account.balance}_i + \text{burned_supply} = \text{total_supply}.$$

13.2. Burn Function

Burn units are defined by

$$\text{burn_units} = 10 + \text{request_kind_units} + \text{proof_system_units} + 4 \cdot \text{tx_count} + \left\lceil \frac{\text{metered_request_size_bytes}}{32} \right\rceil$$

The request-kind and proof-system contributions are:

$$\begin{aligned} \text{request_kind_units}(\text{execution}) &= 5, & \text{request_kind_units}(\text{attestation}) &= 2, \\ \text{proof_system_units}(\text{stark}) &= 3, & \text{proof_system_units}(\text{mock}) &= 1. \end{aligned}$$

13.3. Fail-Closed Burn Semantics

Full burn is consumed on every terminal outcome:

- Accepted,
- ExecutionRejected,
- VerificationRejected,
- SettlementRejected.

This property matters because it removes a class of incentive asymmetry. If some rejected outcomes consumed only partial burn while others consumed full burn, callers could be rewarded for probing ambiguous edges of the protocol. Aura instead ensures that terminal use of the settlement path has deterministic cost semantics independent of whether the final outcome is acceptance or typed rejection.

Theorem 13.1 (Terminal Burn Invariance). *Every terminal local settlement outcome consumes the full burn amount determined by the canonical burn function.*

Proof. The fail-closed burn rule explicitly assigns full burn consumption to all terminal outcomes listed above. Since partial-burn attempts are invalid by contract, no accepted terminal outcome admits a smaller burn. $\square$

14. Continuous Settlement Head Progression

Aura also defines a deterministic local continuous settlement head model. Its key feature is that head linkage is derived, not supplied.

14.1. Canonical Head Transition Input

A canonical transition consumes exactly:

- `prior_head`,
- `settlement_report`.

It does *not* accept caller-supplied `previous_head_hash_hex` or `head_sequence_number` as canonical input.

14.2. Derived Transition Rule

A valid transition must:

1. use `settlement_head_version = 1`,
2. derive

$$\text{previous_head_hash_hex} = \text{prior_head.current_head_hash_hex},$$
3. derive

$$\text{head_sequence_number} = \text{prior_head.head_sequence_number} + 1,$$
4. produce one `current_head_hash_hex`,
5. produce one `canonical_head_commitment_hex`.

14.3. Meaning of the Construction Rule

Any object that directly supplies previous-head linkage fields is non-canonical. This has an important formal consequence.

Proposition 14.1 (Unrepresentability of Canonical Head Drift). *Inside canonical settlement construction, sequence mismatch and previous-head mismatch are unrepresentable as caller-controlled values.*

Proof. The relevant fields are derived from `prior_head` and therefore are not accepted as free canonical inputs. A caller cannot present an alternate canonical transition that differs only by those linkage fields, because such an object is non-canonical by definition. $\square$

This is stronger than merely checking supplied linkage fields against expectations. The protocol does not ask whether a supplied previous-head value is correct; it refuses to admit such a value as part of the canonical construction path at all.

15. UDOT as a Derived Human-Readable Surface

Aura includes a derived glyph-based presentation layer called UDOT. This layer is useful, but its role must be stated precisely to avoid confusion.

15.1. Role of UDOT

UDOT is a derived identity surface. It is *not* the primary cryptographic commitment surface. It must never be treated as an independent canonical proof reference parallel to `proof_hash_hex`.

15.2. Canonical Version and Derivation

Canonical UDOT is fixed to version 2. Given 32-byte `proof_hash_bytes`, define:

`seal_line` = first 16 nibbles of `SHA256("AURA_UDOT_SEAL_LINE_V1" || proof_hash_bytes)`,

`crest` = first 8 nibbles of `SHA256("AURA_UDOT_SEAL_V1" || proof_hash_bytes)`,

`matrix` = first 64 nibbles of `SHA256("AURA_UDOT_MATRIX_V1" || proof_hash_bytes)`.

These nibbles are mapped through the fixed 16-glyph alphabet

◦, , , , , , •, , , , , , , , .

The canonical UDOT bundle is:

- `proof_hash_hex`,
- `seal_line`,
- `crest`,
- `matrix_sequence`.

`matrix_form` is a presentation rendering derived from `matrix_sequence`; it is not canonical wire data. Likewise, `aura_hash_hex` is not a canonical field of the active UDOT bundle.

15.3. Why UDOT Is Useful Without Becoming Truth

Human-readable derived artifacts can aid manual verification, display, indexing, and communication. But the line between derived readability and canonical truth must remain sharp. Aura keeps that boundary clear in two ways:

1. UDOT is derived only from `proof_hash_hex`, never accepted as an equal independent reference surface.
2. Validators that check UDOT recompute it from the canonical proof hash rather than trusting the glyph string on its own.

Remark 15.1. This design allows Aura to expose a recognizable human-facing artifact without sacrificing the single-path property of the canonical pipeline.

16. Formal Model of the Canonical Pipeline

Aura can now be expressed more abstractly.

Let $\mathcal{R}$ denote the set of canonical requests admitted at pipeline entry. Let $\mathcal{A}_i$ denote the set of accepted canonical artifacts at stage i . The active canonical stages are ordered:

$$\mathcal{R} \rightarrow \mathcal{A}_1 \rightarrow \mathcal{A}_2 \rightarrow \cdots \rightarrow \mathcal{A}_9,$$

corresponding respectively to:

`request`, `proof_material_hash`, `proof_hash`, `udot_bundle_v2`, `authorization`, `storm`, `trace`, `proof`, `settle`

For each stage there exists a deterministic map

$$f_i : \mathcal{A}_{i-1} \rightarrow \mathcal{A}_i,$$

where $\mathcal{A}_0 = \mathcal{R}$.

Definition 16.1 (Pure Stage Map). *A stage map f_i is pure if for every accepted input artifact $x \in \mathcal{A}_{i-1}$, the output $f_i(x)$ is uniquely determined and contains no optional or parallel canonical representation of another active concept.*

Aura’s purity rule states that every active stage map is pure.

Define the total canonical pipeline map

$$\Pi = f_9 \circ f_8 \circ \cdots \circ f_1.$$

Theorem 16.1 (Single-Path Pipeline Property). *For every accepted canonical request $r \in \mathcal{R}$, the artifact tuple*

$$(f_1(r), f_2(f_1(r)), \dots, \Pi(r))$$

is uniquely determined.

Proof. Each stage map is pure and deterministic. The composition of deterministic functions is deterministic. Since the canonical path excludes optional parallel representations, there is no second

accepted canonical artifact tuple corresponding to the same accepted request under the active stage ordering. $\square$

This theorem expresses Aura’s most important architectural property. The protocol is not simply deterministic at the recurrence layer; it is deterministic across all canonical artifact boundaries.

17. Core Invariants

Aura’s invariants are best understood not as implementation suggestions but as semantic locks. Violating any one of them changes the protocol.

17.1. Identity Invariants

- `AURA_HASH_V1` is the only identity function.
- Canonical message roots use exactly one `u64_le` length prefix.
- Text mode is NFC plus LF normalization with BOM rejection only.

17.2. Storm and Trace Invariants

- Field elements are canonical 66-byte values below $2^{521} - 1$.
- `STORM_V1_1` uses only the fixed domain-separated derivations.
- `TRACE_ROOT` is the ordered SHA3-256 Merkle root of the full trace.
- `StormClaim521V1` reproduces the derived initial state, final state, and `TRACE_ROOT`.
- `StormPublicInputs521V1` binds side hashes, context hash, boundary states, and `TRACE_ROOT`.

17.3. Pipeline Invariants

- `proof_hash_hex`, `intent_id_hex`, and `proof_session_id_hex` are canonical lowercase 64-hex.
- Each canonical object owns exactly one representation of each concept.
- Downstream canonical objects reference upstream artifacts by `proof_hash_hex`; they do not embed exact nested copies of upstream canonical objects.
- Canonical UDOT is v2-only and derived from `proof_hash_hex`.
- Canonical authorization lineage has exactly one six-field encoding.
- Legacy normalization and version selection happen before canonical pipeline entry.
- Unknown wire fields are rejected.

17.4. Ledger and Settlement Invariants

- Ledger supply balances exactly with burned supply.
- The payer account exists before burn.
- Full burn is consumed on every terminal outcome.
- Head sequence is derived from the prior head.
- Previous head hash is derived from the prior head.

These invariants function together. The protocol is strongest not because any single rule is novel, but because the rules are coordinated so that ambiguity removed at one stage does not reappear at another.

18. Failure Classes and Fail-Closed Validation

Aura classifies failure rather than softening it. The active failure classes include:

- HASH_INPUT_INVALID,
- HASH_TEXT_INVALID,
- FIELD_ENCODING_INVALID,
- STORM_CONTEXT_INVALID,
- STORM_CLAIM_INVALID,
- TRACE_COMMITMENT_INVALID,
- PROVER_BINDING_INVALID,
- UDOT_INVALID,
- PIPELINE_WIRE_INVALID,
- AUTHORIZATION_INVALID,
- LEDGER_INVALID,
- BURN_INVALID,
- SETTLEMENT_INVALID.

What is equally important is what does *not* exist. Aura excludes failure classes for duplicated representations, equivalence mismatch, and cross-representation drift in cases where the canonical protocol does not admit such duplicated representations in the first place. Similarly, a distinct HEAD_INVALID class is retired because canonical head construction does not admit independently encoded previous-head or sequence fields.

Definition 18.1 (Fail-Closed Validation). *Validation is fail closed if malformed, ambiguous, or unsupported input is rejected rather than normalized into a canonical form inside the active path.*

Aura implements fail-closed validation at every layer. Canonical parsers reject unknown fields. Settlement-wire parsing rejects omitted required fields. UDOT parsers distinguish syntax from semantic verification. Non-canonical hex spelling is not silently repaired. These choices are all manifestations of the same doctrine.

19. Security Analysis

Aura’s security properties are best understood as a layered consequence of canonical exclusivity and deterministic construction.

19.1. Collision Resistance and Identity Integrity

The primary identity function uses SHA-256 with a fixed domain separator and exact framing. Therefore the security of the canonical message root reduces to the collision and second-preimage resistance of that construction together with the uniqueness of the framing rule. In practice, this means that an attacker seeking two accepted canonical payloads with the same message root must either find a collision in the underlying hash or exploit a second accepted canonical framing path. Aura removes the latter possibility by design.

19.2. Field and State Encoding Integrity

Field encoding attacks are limited by the fixed 66-byte canonical form, range checks, and top-bit constraint. An attacker cannot exploit width variation or alternative padding to alter row bytes while preserving field semantics, because the protocol does not admit alternate canonical field encodings.

19.3. Trace Binding Integrity

The trace commitment binds both content and order. Since `TRACE_ROOT` is derived from the ordered full trace and no row may be inferred or omitted, an attacker cannot replace the committed execution with a shorter, reordered, or partially reconstructed trace without altering the root, except by breaking the underlying hash assumption.

19.4. Proof Binding Integrity

The proof layer is protected not only by the soundness of the STARK system but by the narrowness of the canonical proof boundary. Proof verification is bound to one canonical claim representation and one compact public input structure. This blocks ambiguity that might otherwise arise from mixed legacy and current claim encodings or from proof objects that redundantly embed upstream context.

19.5. Authorization and Replay Resistance

The authorization lineage binds subject, intent, freshness, and proof reference under a fixed six-field encoding. Because freshness and subject bindings are explicit and because downstream stages reference only the canonical proof hash rather than embed parallel objects, replay or reinterpretation attempts must attack the canonical proof reference or the fixed binding fields themselves. There is no canonical second path through an alternate embedded object.

19.6. Settlement and Head Integrity

Settlement-wire minimality ensures that the final externally transported object cannot carry contradictory upstream canonical state. Continuous head construction is further protected because previous-head linkage and sequence are derived from prior state rather than caller supplied. This removes an entire attack category based on crafting well-formed but semantically conflicting linkage fields.

19.7. Presentation-Layer Safety

UDOT is intentionally limited. Its glyph strings are deterministic derivatives of the proof hash, but they are not promoted to independent canonical truth. This prevents display or formatting artifacts from competing with the proof hash as a cryptographic identity surface.

19.8. Threat Summary

The main attacker goals and Aura’s responses may be summarized as follows.

Attacker Goal	Aura Response
Exploit alternate message framing or normalization	One identity function, one framing rule, one narrow text helper profile
Exploit alternate field encodings	Fixed 66-byte canonical encoding and range checks
Alter trace order or omit rows	Ordered row layout and ordered Merkle commitment
Bind a proof to parallel claim representations	Single canonical storm claim and single proof envelope boundary
Replay or reinterpret proof under alternate authorization metadata	Fixed six-field authorization lineage with canonical proof-hash reference
Inject contradictory upstream objects at settlement boundary	Final settlement wire carries only canonical proof reference
Forge settlement linkage	Previous-head and sequence are derived from prior head, not free inputs
Treat human-readable surface as canonical truth	UDOT is derived presentation, never an equal canonical reference

20. Implementation Consequences

Aura’s design is demanding in implementation terms, because convenience patterns common in many codebases become invalid. The following consequences are intentional.

20.1. No Silent Repair

Parsers must reject rather than auto-correct:

- uppercase canonical hex where lowercase is required,
- omitted required fields,

- unknown wire fields,
- whitespace-polluted canonical glyph strings,
- malformed matrix rendering,
- non-canonical field encodings.

20.2. No Parallel Object Carrying

Developers often prefer to keep full upstream objects nested in later objects “for convenience.” Aura rejects that approach along the canonical path. Later objects should carry references, not re-embed canonical upstream objects.

20.3. No Partial Canonicity

An implementation is not canonical merely because its high-level outputs look equivalent. It must match byte-level contracts: message framing, field encoding, row formation, Merkle construction, proof envelope structure, authorization lineage structure, UDOT derivation, and settlement-wire exactness.

20.4. Validation Matrix as an Engineering Requirement

Aura’s frozen validation surface spans:

- L0 hash and text canonicalization vectors,
- L1–L2 storm execution, trace commitment, claim, and hardening vectors,
- L3–L5 canonical prepare, pipeline, and UDOT vectors,
- local settlement and continuous-chain fixtures.

This cross-language vectorization is not optional ceremony. It is the practical mechanism by which single-path semantics remain single-path under real implementation pressure.

21. Conceptual Position of Aura

Aura should not be understood merely as “a system that uses hashes, field arithmetic, and STARKs.” Those are ingredients. The distinctive architectural claim is stronger:

Aura is a protocol in which every active canonical artifact is the unique downstream consequence of one unique upstream canonical interpretation.

That claim changes how the system should be evaluated. One should not ask only whether the proof system is sound, or whether the hash function is collision resistant, or whether the settlement head can be verified after the fact. One should also ask whether the protocol structure itself admits parallel accepted representations at any stage. Aura attempts to answer no.

This is why the protocol insists on concept ownership, strict document ordering, fail-closed compatibility boundaries, minimal settlement references, and derivation of authority-critical linkage fields. These decisions may appear strict when viewed independently. Taken together, they define the protocol.

22. Conclusion

Aura presents a deterministic commitment-native protocol centered on one idea: canonical ambiguity should be removed structurally, not merely checked procedurally. The system therefore fixes one message identity function, one field encoding, one family of domain-separated derivations, one deterministic recurrence, one ordered trace commitment, one canonical proof-bound claim representation, one artifact derivation chain, one canonical authorization lineage encoding, one minimal settlement-wire reference object, one local burn function with terminal full-burn semantics, and one derived continuous-head construction that excludes caller-controlled linkage fields.

The resulting architecture is single path. For accepted canonical inputs, later pipeline artifacts are uniquely determined. For accepted canonical outputs, upstream interpretation is uniquely constrained by the active protocol contracts. This does not eliminate the need for strong primitives; rather, it ensures that strong primitives are not embedded inside a semantically loose protocol shell.

Aura is therefore best viewed as a formal commitment pipeline with proof and settlement surfaces, not as a generic computation framework with optional canonicalization. Its contribution lies in turning exactness into protocol structure. Identity is singular. Execution is deterministic. Commitment is ordered. Proof binding is exclusive. Authorization is fixed. Settlement is minimal. Head progression is derived. Human-readable representation remains derived. Failure is typed and fail closed.

In that sense, Aura advances a broader design principle for verifiable systems: when canonical structure is treated as a first-class security boundary, the protocol becomes not only easier to validate, but harder to misinterpret.